



Data Engineering

MINI PROJECT



Sales Data Warehouse



Mini Project: Sales Data Warehouse & Business Reporting

Project Title - SQL for Data Engineers (PostgreSQL)

Project Objective

Design and build a simple **Sales Data Warehouse** using PostgreSQL. You will create a sales data model, implement a star schema, load data into the warehouse, and generate business insights using SQL reporting queries.

Business Scenario

A retail company operates multiple stores across different cities.

The company sells products belonging to various categories such as:

-  Electronics
-  Clothing
-  Groceries
-  Home Goods

Management wants a centralized reporting system to answer questions such as:

- What are our total sales?
- Which products generate the highest revenue?
- Which customers spend the most money?
- Which stores perform best?
- How are sales changing month over month?
- Which product categories are growing?
- Which cities generate the most revenue?

The current transactional database is optimized for daily operations but not for analytics.

As a Data Engineer, your task is to create a Data Warehouse that supports fast reporting and business intelligence.

Architecture

Source Systems

|



Operational Database (OLTP)

|



ETL / ELT Process

|



Dimensional Data Warehouse

|



Business Reports & Dashboards

Step 1: Design the Operational Data Model

The operational system stores daily transactions.

Customers

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50),  
  email VARCHAR(100),  
  city VARCHAR(50),
```

```
country VARCHAR(50)
);
```

Products

```
CREATE TABLE products (
  product_id SERIAL PRIMARY KEY,
  product_name VARCHAR(100),
  category VARCHAR(50),
  price NUMERIC(10,2)
);
```

Stores

```
CREATE TABLE stores (
  store_id SERIAL PRIMARY KEY,
  store_name VARCHAR(100),
  city VARCHAR(50)
);
```

Orders

```
CREATE TABLE orders (
  order_id SERIAL PRIMARY KEY,
  customer_id INT,
  store_id INT,
  order_date DATE
);
```

Order Details

```
CREATE TABLE order_details (
  order_detail_id SERIAL PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  unit_price NUMERIC(10,2)
);
```

Why Not Use This Model for Reporting?

This model is highly normalized.

To answer a simple question such as:

💰 "What were monthly sales by category?"

The database must join multiple tables.

```
orders
|
order_details
|
products
|
customers
|
stores
```

```
SELECT
    DATE_TRUNC('month', o.order_date) AS sales_month,
    p.category,
    SUM(od.quantity * od.unit_price) AS total_sales
FROM orders o
JOIN order_details od
    ON o.order_id = od.order_id
JOIN products p
    ON od.product_id = p.product_id
GROUP BY
    DATE_TRUNC('month', o.order_date),
    p.category
ORDER BY
    sales_month,
    p.category;
```

As data grows to millions of rows, reporting becomes slow.

Therefore we build a Data Warehouse.

★ Step 2: Design the Star Schema

A Star Schema is optimized for analytical workloads.

It contains:

Fact Tables

Contain measurable business events.

Examples:

- Sales Amount
- Quantity Sold
- Profit

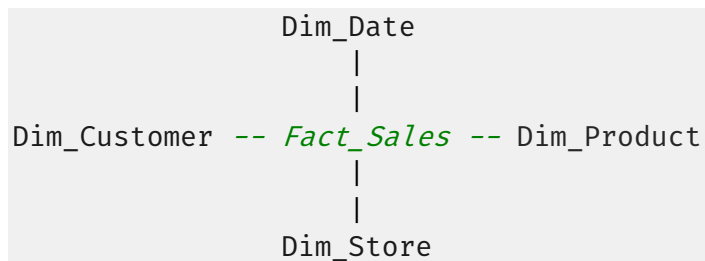
Dimension Tables

Contain descriptive information.

Examples:

- Customer
- Product
- Store
- Date

🌟 Star Schema Design





Date Dimension

One of the most important dimensions.

```
CREATE TABLE dim_date (  
    date_key INT PRIMARY KEY,  
    full_date DATE,  
    day INT,  
    month INT,  
    month_name VARCHAR(20),  
    quarter INT,  
    year INT,  
    week_number INT  
);
```



Customer Dimension

```
CREATE TABLE dim_customer (  
    customer_key SERIAL PRIMARY KEY,  
    customer_id INT,  
    customer_name VARCHAR(100),  
    city VARCHAR(50),  
    country VARCHAR(50)  
);
```



Product Dimension

```
CREATE TABLE dim_product (  
    product_key SERIAL PRIMARY KEY,  
    product_id INT,  
    product_name VARCHAR(100),  
    category VARCHAR(50)  
);
```



Store Dimension

```
CREATE TABLE dim_store (  
    store_key SERIAL PRIMARY KEY,  
    store_id INT,  
    store_name VARCHAR(100),  
    city VARCHAR(50)  
);
```

Fact Sales Table

This is the center of the warehouse.

```
CREATE TABLE fact_sales (  
    sales_key BIGSERIAL PRIMARY KEY,  
    date_key INT,  
    customer_key INT,  
    product_key INT,  
    store_key INT,  
    quantity INT,  
    sales_amount NUMERIC(12,2)  
);
```

Step 3: Build the ETL Process

Extract

Read data from source tables.

```
SELECT * FROM orders;  
SELECT * FROM order_details;
```

Transform

Create business metrics.

```
quantity * unit_price AS sales_amount
```

Load Dimensions

Customer Dimension

```
INSERT INTO dim_customer  
(  
    customer_id,  
    customer_name,  
    city,  
    country  
)  
SELECT  
    customer_id,  
    first_name || ' ' || last_name,  
    city,
```

```
country
FROM customers;
```

Product Dimension

```
INSERT INTO dim_product
(
    product_id,
    product_name,
    category
)
SELECT
    product_id,
    product_name,
    category
FROM products;
```

Store Dimension

```
INSERT INTO dim_store
(
    store_id,
    store_name,
    city
)
SELECT
    store_id,
    store_name,
    city
FROM stores;
```

Load Fact Table

```
INSERT INTO fact_sales
(
    date_key,
    customer_key,
    product_key,
    store_key,
    quantity,
    sales_amount
)
SELECT
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
```

```

        dc.customer_key,
        dp.product_key,
        ds.store_key,
        od.quantity,
        od.quantity * od.unit_price
FROM orders o
JOIN order_details od
    ON o.order_id = od.order_id
JOIN dim_customer dc
    ON o.customer_id = dc.customer_id
JOIN dim_product dp
    ON od.product_id = dp.product_id
JOIN dim_store ds
    ON o.store_id = ds.store_id;

```

✅ Step 4: Data Quality Validation

Data Engineers always validate loaded data.

Row Count Check

```

SELECT COUNT(*)
FROM fact_sales;

```

Null Check

```

SELECT *
FROM fact_sales
WHERE sales_amount IS NULL;

```

Duplicate Check

```

SELECT
    date_key,
    customer_key,
    product_key,
    COUNT(*)
FROM fact_sales
GROUP BY
    date_key,
    customer_key,
    product_key
HAVING COUNT(*) > 1;

```

Business Reporting Queries

Total Revenue

```
SELECT
    SUM(sales_amount) AS total_revenue
FROM fact_sales;
```

Monthly Sales Trend

```
SELECT
    d.year,
    d.month_name,
    SUM(f.sales_amount) revenue
FROM fact_sales f
JOIN dim_date d
ON f.date_key=d.date_key
GROUP BY
    d.year,
    d.month,
    d.month_name
ORDER BY
    d.year,
    d.month;
```

Top 10 Products

```
SELECT
    p.product_name,
    SUM(f.sales_amount) revenue
FROM fact_sales f
JOIN dim_product p
ON f.product_key=p.product_key
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 10;
```

Top Customers

```
SELECT
    c.customer_name,
    SUM(f.sales_amount) total_spent
FROM fact_sales f
JOIN dim_customer c
ON f.customer_key=c.customer_key
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 10;
```

Best Performing Stores

```
SELECT
    s.store_name,
    SUM(f.sales_amount) revenue
FROM fact_sales f
JOIN dim_store s
ON f.store_key=s.store_key
GROUP BY s.store_name
ORDER BY revenue DESC;
```

Sales by Category

```
SELECT
    p.category,
    SUM(f.sales_amount) revenue
FROM fact_sales f
JOIN dim_product p
ON f.product_key=p.product_key
GROUP BY p.category
ORDER BY revenue DESC;
```

Advanced SQL for Data Engineers

Running Total

```
SELECT
    d.full_date,
    SUM(f.sales_amount) daily_sales,
    SUM(SUM(f.sales_amount))
    OVER(
```

```

        ORDER BY d.full_date
    ) running_total
FROM fact_sales f
JOIN dim_date d
ON f.date_key=d.date_key
GROUP BY d.full_date;

```

Month-over-Month Growth

```

WITH monthly_sales AS
(
SELECT
    d.year,
    d.month,
    SUM(f.sales_amount) revenue
FROM fact_sales f
JOIN dim_date d
ON f.date_key=d.date_key
GROUP BY d.year,d.month
)

SELECT
    year,
    month,
    revenue,
    LAG(revenue)
OVER(ORDER BY year,month)
previous_month,
ROUND(
    (
        revenue -
        LAG(revenue)
        OVER(ORDER BY year,month)
    )
    /
    LAG(revenue)
    OVER(ORDER BY year,month)
    *100,
    2
) growth_pct
FROM monthly_sales;

```

Customer Segmentation

```
SELECT
    customer_name,
    SUM(sales_amount) total_spent,
    CASE
        WHEN SUM(sales_amount) > 10000
            THEN 'VIP'
        WHEN SUM(sales_amount) > 5000
            THEN 'Gold'
        ELSE 'Regular'
    END customer_segment
FROM fact_sales f
JOIN dim_customer c
ON f.customer_key=c.customer_key
GROUP BY customer_name;
```

Performance Optimization

Create indexes for faster analytics.

```
CREATE INDEX idx_fact_sales_date
ON fact_sales(date_key);

CREATE INDEX idx_fact_sales_product
ON fact_sales(product_key);

CREATE INDEX idx_fact_sales_customer
ON fact_sales(customer_key);
```

Learning Outcomes

After completing this project, students will understand:

Data Modeling

- ✓ OLTP vs OLAP
- ✓ Normalized Models
- ✓ Dimensional Modeling

Data Warehousing

- ✓ Fact Tables
- ✓ Dimension Tables
- ✓ Star Schema Design

ETL / ELT

- ✓ Extract Data
- ✓ Transform Data
- ✓ Load Data

SQL Engineering

- ✓ Joins
- ✓ Aggregations
- ✓ Window Functions
- ✓ CTEs
- ✓ Data Validation Queries

Analytics Engineering

- ✓ Revenue Reporting
- ✓ Customer Analytics
- ✓ Product Analytics
- ✓ Store Performance Analysis

Data Engineering Best Practices

- ✓ Surrogate Keys
- ✓ Data Quality Checks
- ✓ Performance Tuning
- ✓ Scalable Warehouse Design